Path 2: Cyber Security 101 > Module 2: Linux Fundamentals > Room 1: Linux Fundamentals Part 1

https://tryhackme.com/room/linuxfundamentalspart1

Linux Fundamentals Part 1:
Embark on the journey of learning the fundamentals of Linux. Learn to run some of the first essential commands on an interactive terminal.

>> Task 1: Introduction

Welcome to the first part of the "Linux Fundamentals" room series. You're most likely using a Windows or Mac machine, both are different in visual design and how they operate. Just like Windows, iOS and MacOS, Linux is just another operating system and one of the most popular in the world powering smart cars, android devices, supercomputers, home appliances, enterprise servers, and more.

>> Task 2: A Bit of Background on Linux

Where is Linux Used?

It's fair to say that Linux is a lot more intimidating to approach than Operating System's (OSs) such as Windows. Both variants have their own advantages and disadvantages. For example, Linux is considerably much more lightweight and you'd be surprised to know that there's a good chance you've used Linux in some form or another every day! Linux powers things such as:

- Websites that you visit
- Car entertainment/control panels
- Point of Sale (PoS) systems such as checkout tills and registers in shops
- Critical infrastructures such as traffic light controllers or industrial sensors

Flavours of Linux:

The name "Linux" is actually an umbrella term for multiple OS's that are based on UNIX (another operating system). Thanks to Linux being open-source, variants of Linux come in all shapes and sizes - suited best for what the system is being used for.

For example, Ubuntu & Debian are some of the more commonplace distributions of Linux because it is so extensible. I.e. you can run Ubuntu as a server (such as websites & web applications) or as a fully-fledged desktop. For this series, we're going to be using Ubuntu.

Note: Ubuntu Server can run on systems with only 512MB of RAM!

Similar to how you have different versions Windows (7, 8 and 10), there are many different versions/distributions of Linux.

Q1) Research: What year was the first release of a Linux operating system?
Answer: 1991

>> Task 3: Interacting With Your First Linux Machine (In-Browser)

I've deployed my first Linux machine!

>> Task 4: Running Your First few Commands

As we previously discussed, a large selling point of using OSs such as Ubuntu is how lightweight they can be. This, of course, doesn't come without its disadvantages, where for example, often there is no GUI (Graphical User Interface) or what is also known as a desktop environment that we can use to interact with the machine (unless it has been installed). A large part of interacting with these systems is using the "Terminal". The "Terminal" is purely text-based and is intimidating at first.

Command: echo
Description: Output any text that we provide
Syntax: echo <content>
echo Hello
echo "Hello Friend!"

if we want to echo a single word, we don't need to use double quotes, for example, echo Hello. However, the string should be enclosed within double quotes if one or more spaces are present, for example, echo "Hello Friend!".

Command: whoami
Description: Find out what user we're currently logged in as!
Syntax: whoami

whoami can be used to find the username we are logged in as.

Q1) If we wanted to output the text "**TryHackMe**", what would our command be?
Answer: echo TryHackMe

Q2) What is the username of who you're logged in as on your deployed Linux machine?
Answer: tryhackme

>> Task 5: Interacting With the Filesystem!

Interacting With the Filesystem:

Command: ls
Full Name: listing

Command: cd
Full Name: change directory

Command: cat
Full Name: concatenate

Command: pwd
Full Name: print working directory

- Listing Files in Our Current Directory (ls):

Before we can do anything such as finding out the contents of any files or folders, we need to know what exists in the first place. This can be done using the "ls" command (short for listing)

Pro tip: You can list the contents of a directory without having to navigate to it by using ls and the name of the directory. I.e. ls <FolderName>

- Changing Our Current Directory (cd):

Now that we know what folders exist, we need to use the "cd" command (short for **c**hange **d**irectory) to change to that directory. Say if I wanted to open the "Pictures" directory - I'd do "cd Pictures". Where again, we want to find out the contents of this "Pictures" directory and to do so, we'd use "ls" again:

- Outputting the Contents of a File (cat):

Whilst knowing about the existence of files is great — it's not all that useful unless we're able to view the contents of them.

We will come on to discuss some of the tools available to us that allows us to transfer files from one machine to another in a later room. But for now, we're going to talk about simply seeing the contents of text files using a command called "cat".

"Cat" is short for concatenating & is a fantastic way for us to output the contents of files (not just text files!).

Pro tip: You can use cat to output the contents of a file within directories without having to navigate to it by using cat and the name of the directory. I.e. cat /home/ubuntu/Documents/todo.txt

- Finding out the full Path to our Current Working Directory (pwd):

You'll notice as you progress through navigating your Linux machine, the name of the directory that you are currently working in will be listed in your terminal.

It's easy to lose track of where we are on the filesystem exactly, which is why I want to introduce "pwd". This stands for **p**rint **w**orking **d**irectory.

Q1) On the Linux machine that you deploy, how many folders are there?
Answer: 4

Q2) Which directory contains a file?
Answer: folder4

Q3) What is the contents of this file?
Answer: Hello World!

Q4) Use the cd command to navigate to this file and find out the new current working directory. What is the path?
Answer: /home/tryhackme/folder4

>> Task 6: Searching for Files

🔹 Using Find:

The find command is fantastic in the sense that it can be used both very simply or rather complex depending upon what it is you want to do exactly. However, let's stick to the fundamentals first.

Command: find

To find Directories/Files -
Syntax: find -name <File/FolderName>

Using "find" to find any file with the extension of ".txt" -
find -name *.txt

🔹 Using Grep:

Another great utility that is a great one to learn about is the use of grep. The grep command allows us to search the contents of files for specific values that we are looking for.

Take for example, the access log of a web server. In this case, the access.log of a web server has 244 entries.

Command: wc
Syntax: wc <FileName>

- First number → Number of lines
- Second number → Number of words
- Third number → Number of bytes (file size in characters)
- Filename → The file you checked

✅ Use Case
- Gives you a quick overview of the size and content density of a file.
- For log files, the line count is usually the most useful (each line = one request).

- If you only want one metric, you can add options:
- wc -l <FileName> → lines only
- wc -w <FileName> → words only
- wc -c <FileName> → bytes only

Command: grep
Syntax: grep "pattern" <FileName>

- Searching Recursively with grep:

Sometimes, the information we are looking for is spread across multiple files inside a directory. Instead of checking each file individually, we can tell grep to search **recursively** through all files and subdirectories.

To do this, we use the -R (recursive) option.

For example, to search for a variable across **all files** in the current directory and its subfolders, we can run:

```
grep -R "PRETTY_NAME" /etc/
```

This will:
- Search every file in the current directory
- Search all subdirectories
- Show where the PRETTY_NAME appears

Example output:

```
grep -R "PRETTY_NAME" /etc/
grep: /etc/sudoers: Permission denied
/etc/os-release:PRETTY_NAME="Ubuntu"
```

Q1) Use grep on "access.log" to find the flag that has a prefix of "THM". What is the flag? **Note**: The "access.log" file is located in the "/home/tryhackme/" directory.
Answer: THM{ACCESS}

How:
```
grep THM /home/tryhackme/access.log
grep THM* /home/tryhackme/access.log
```

>> Task 7: An Introduction to Shell Operators

Linux operators are a fantastic way to power up your knowledge of working with Linux. There are a few important operators that are worth noting. We'll cover the basics and break them down accordingly to bite-sized chunks.

Symbol / Operator	**Description**
&	This operator allows you to run commands in the background of your terminal.
&&	This operator allows you to combine multiple commands together in one line of your terminal.
>	This operator is a redirector - meaning that we can take the output from a command (such as using cat to output a file) and direct it elsewhere.
>>	This operator does the same function of the > operator but appends the output rather than replacing (meaning nothing is overwritten).

Operator "&":

This operator allows us to execute commands in the background. For example, let's say we want to copy a large file. This will obviously take quite a long time and will leave us unable to do anything else until the file successfully copies.

The "&" shell operator allows us to execute a command and have it run in the background (such as this file copy) allowing us to do other things!

Operator "&&":

This shell operator is a bit misleading in the sense of how familiar is to its partner "&". Unlike the "&" operator, we can use "&&" to make a list of commands to run for example command1 && command2. However, it's worth noting that **command2** will only run if **command1** was successful.

Operator ">":

This operator is what's known as an output redirector. What this essentially means is that we take the output from a command we run and send that output to somewhere else.

A great example of this is redirecting the output of the echo command that we learned in Task 4. Of course, running something such as echo howdy will return "howdy" back to our terminal — that isn't super useful. What we can do instead, is redirect "howdy" to something such as a new file!

Let's say we wanted to create a file named "welcome" with the message "hey". We can run echo hey > welcome where we want the file created with the contents "hey" like so:

Note: If the file i.e. "welcome" already exists, the contents will be overwritten!

Operator ">>":

This operator is also an output redirector like in the previous operator (>) we discussed. However, what makes this operator different is that rather than overwriting any contents within a file, for example, it instead just puts the output at the end.

Following on with our previous example where we have the file "welcome" that has the contents of "hey". If were to use echo to add "hello" to the file using the > operator, the file will now only have "hello" and not "hey".

The >> operator allows to append the output to the bottom of the file — rather than replacing the contents like so:

```
echo hello >> welcome
cat welcome
```

Q1) If we wanted to run a command in the background, what operator would we want to use?
Answer: &
Example: cp largefile.txt /destination/ &

Q2) If I wanted to replace the contents of a file named "passwords" with the word "password123", what would my command be?
Answer: echo password123 > passwords

Q3) Now if I wanted to add "tryhackme" to this file named "passwords" but also keep "passwords123", what would my command be
Answer: echo tryhackme >> passwords